CIS 371 Web Application Programming

HTTP

Lecturer: Dr. Haoyu Li

Based on the original version by Professor Hans Dulimarta and Professor Yong Zhuang.

```html
<div class="alert-box">
<span class="alert-title">Success!</span>
Your changes have been saved.
</div>


<div class="info-box">
<span class="alert-title">Note:</span>
The system will restart in 5 minutes.
</div>
```

Practice: Sass

Answer

```css
.alert-box {
display: flex;
align-items: center;
padding: 15px;
border-radius: 5px;
background-color: #f8d7da;
color: #721c24;
}


.alert-box .alert-title {
font-weight: bold;
font-size: 1.2rem;
margin-right: 10px;
}


.info-box {
display: flex;
align-items: center;
padding: 15px;
border-radius: 5px;
background-color: #d1ecf1;
color: #0c5460;
}
```

Use the online playground to convert the CSS code into Sass. Implement the following features:

- Inheritance: Create a %flex-layout placeholder to share the display, align-items, and padding between both boxes.
- Mixins: Create a @mixin alert-theme($bg, $text) to handle the background and text colors in one line.
- Nesting: Nest the .alert-title inside the .alert-box.
- Operators: Use a $base-unit: 5px; and set the border-radius and margin-right using multiplication.

```html
<div class="alert-box">
  <span class="alert-title">Success!</span>
  Your changes have been saved.
</div>
<div class="info-box">
  <span class="alert-title">Note:</span>
  The system will restart in 5 minutes.
</div>
```

```scss
// Brand Colors
$alert-bg: #f8d7da;
$alert-text: #721c24;
$info-bg: #d1ecf1;
$info-text: #0c5460;
// Operator: Base Spacing Unit
$base-unit: 5px;

// 1. Inheritance: Extract the "Super Style" for layout
%flex-layout {
  display: flex;
  align-items: center;
  padding: $base-unit * 3; // 15px
}

// 2. Mixin: Create a template for the color themes
@mixin box-theme($bg, $text) {
  background-color: $bg;
  color: $text;
}

// 3. Applying the features
.alert-box {
  @extend %flex-layout;
@include box-theme($alert-bg, $alert-text);
  border-radius: $base-unit; // 5px

  // 4. Nesting
  .alert-title {
    font-weight: bold;
    font-size: 1.2rem;
    margin-right: $base-unit * 2; // 10px
  }
}

// 5. Applying the features
.info-box {
  @extend %flex-layout;
  @include box-theme($info-bg, $info-text);
  border-radius: $base-unit;
}
```

```css
.alert-box {
display: flex;
align-items: center;
padding: 15px;
border-radius: 5px;
background-color: #f8d7da;
color: #721c24;
}
.alert-box .alert-title {
font-weight: bold;
font-size: 1.2rem;
margin-right: 10px;
}
.info-box {
display: flex;
align-items: center;
padding: 15px;
border-radius: 5px;
background-color: #d1ec
color: #0c5460;
}
```

When you type http://info.cern.ch/ and press Enter… what actually happens?

DNS?

Server?

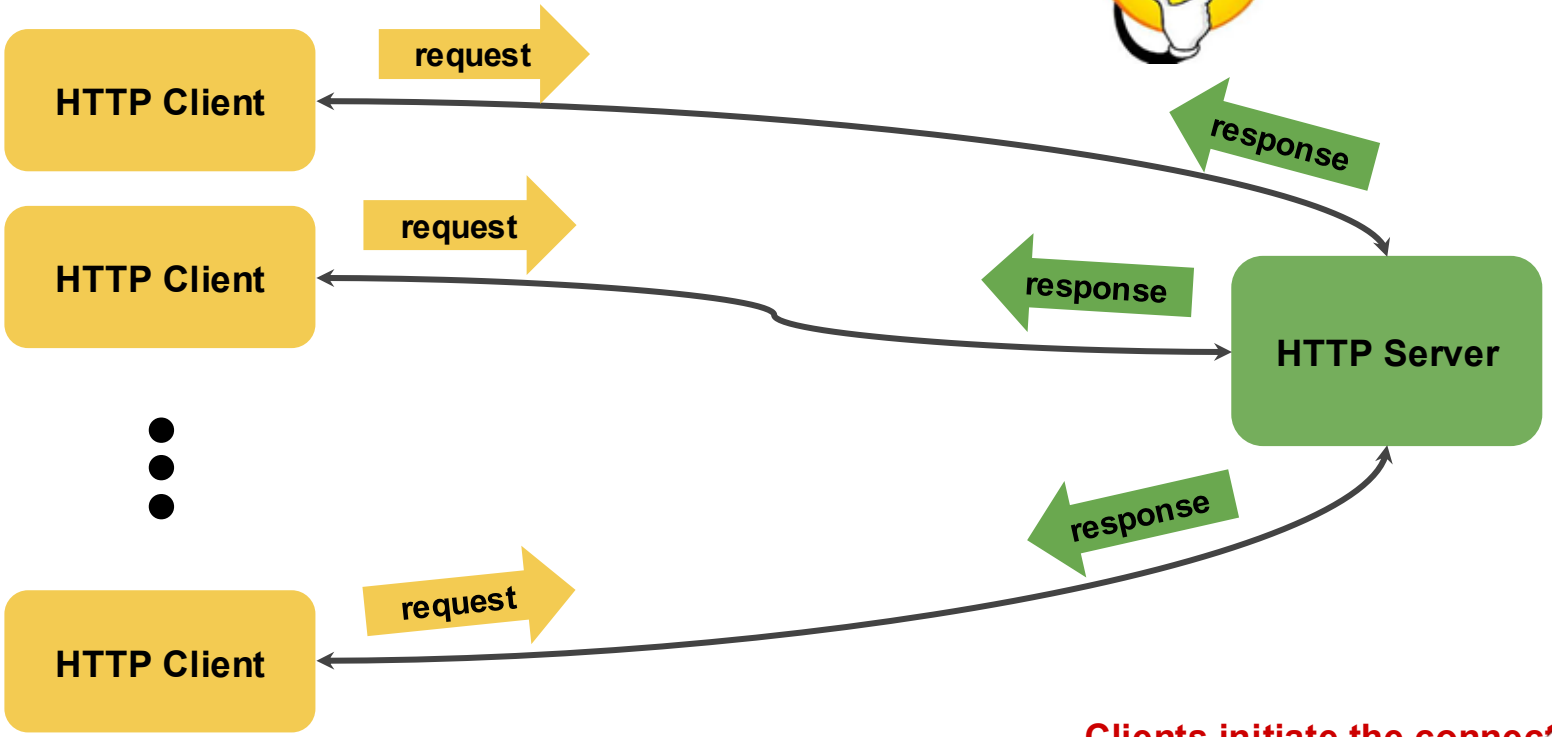
HTML?

TCP?

GET?

HTTP Communication Model

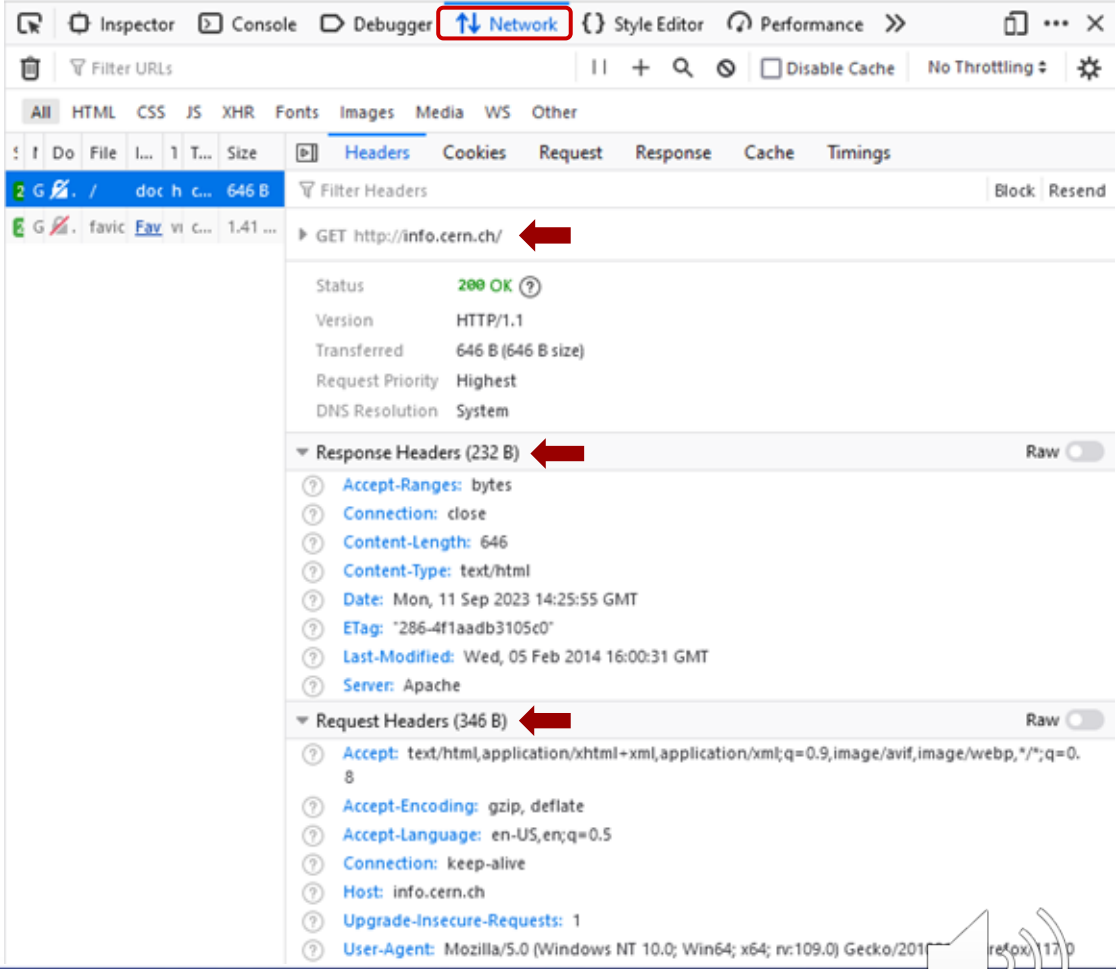
Who initiates?



Clients initiate the connection!!!

Web Browser DevTools (Network Tab)

http://info.cern.ch



http://info.mysite.org/about/

*Notice the **blank line** after the "Accept-Language" header*

```
GET /about HTTP/1.0
Host: info.mysite.org
Accept-Language: en-us
```

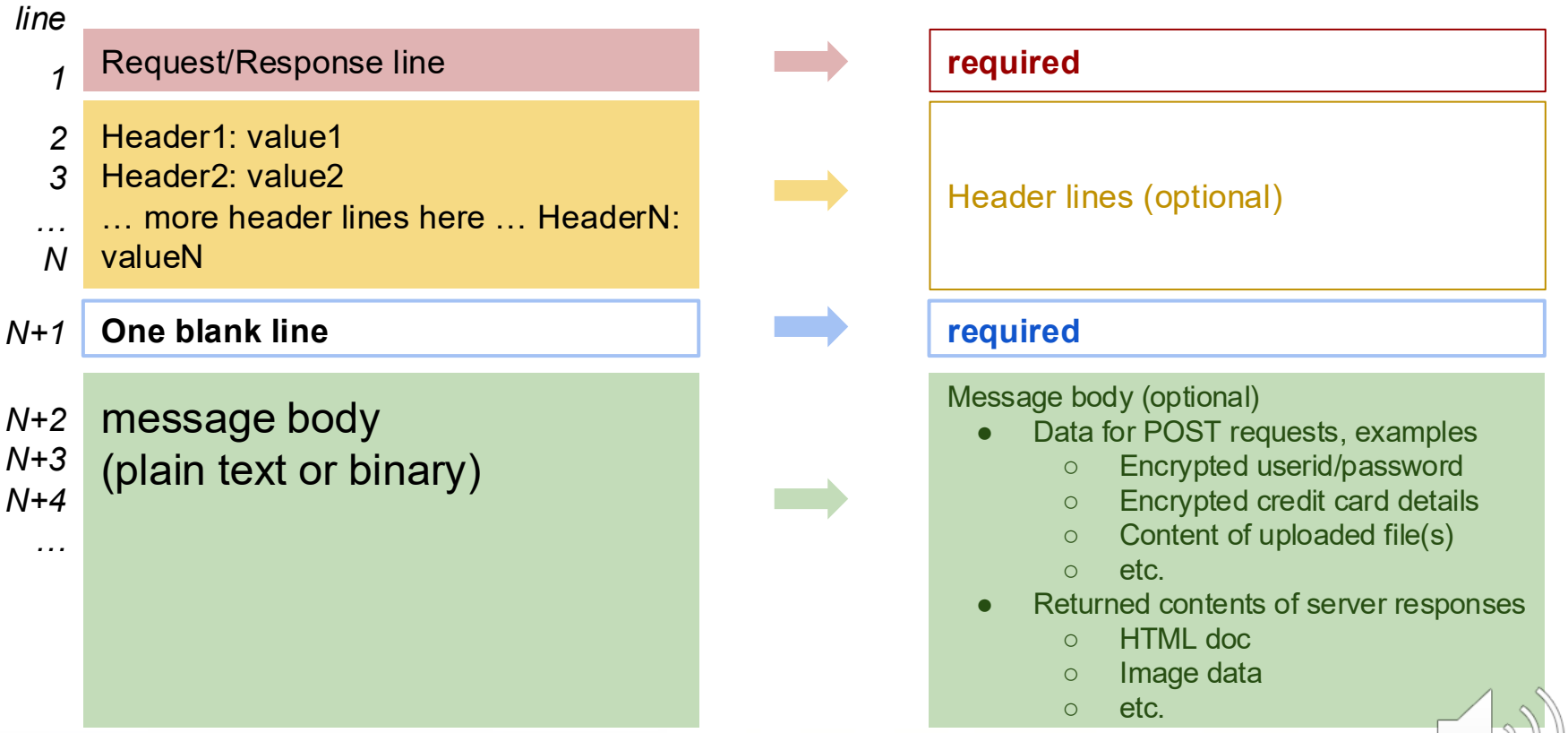
Client

Server (info.mysite.org)

```
HTTP/1.0 200 OK
Content-Type: text/html
Host: info.mysite.org

<html> <head><title>Welcome</title></head>
   <body>
      <h1>To my site</h1>
   </body>
</html>
```

HTTP Request/Response



HTTP headers of interest to web developers

Header	Description	Example
Accept	Inform server media-type to respond	Accept: image/jpg
Accept-Language	Inform the server which languages the client is able to understand	Accept-Language: en-US; en-UK
Content-Type	Media type of the returned content	Content-Type: plain/text
Content-Language	The languages of the content	Content-Language: en-US
Date	Date and time of the message	Date: Mon, 21 Aug 2017 18:14:36 GMT
ETag	Identifier used by caching algorithms	ETag: “"8a9-291e721905000”
Host	Specify the domain name of the intended server (mainly for Virtual Hosting)	Host: www.personal.me:5555

Fetch the content via the command line

```
curl --verbose http://info.cern.ch
```

(On Linux/OSX/Windows 10 WSL)

Fetch the content via the command line

```
iwr http://info.cern.ch -UseBasicParsing
```

(On Windows PowerShell)

CURL

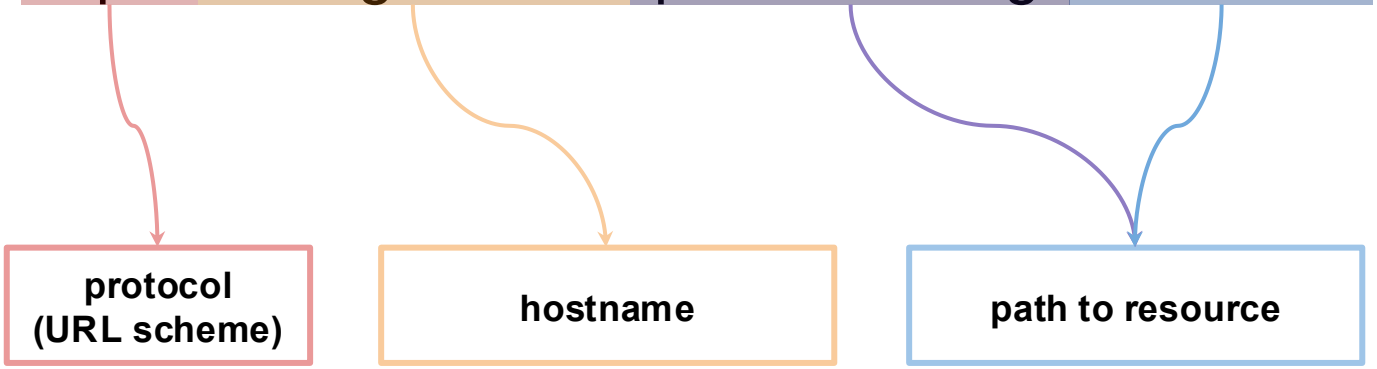
```
(base) ➜  ~ curl --verbose http://info.cern.ch
> GET / HTTP/1.1
> Host: info.cern.ch
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 08 Aug 2026 13:42:31 GMT
< Server: Apache
< Last-Modified: Wed, 05 Feb 2014 16:00:31 GMT
< ETag: "286-4f1aadb3105c0"
< Accept-Ranges: bytes
< Content-Length: 646
< Connection: close
< Content-Type: text/html
<
<html><head></head><body><header>
<title>http://info.cern.ch</title>
</header>
……
</body></html>
```

HTTP URL: Uniform Resource Locator

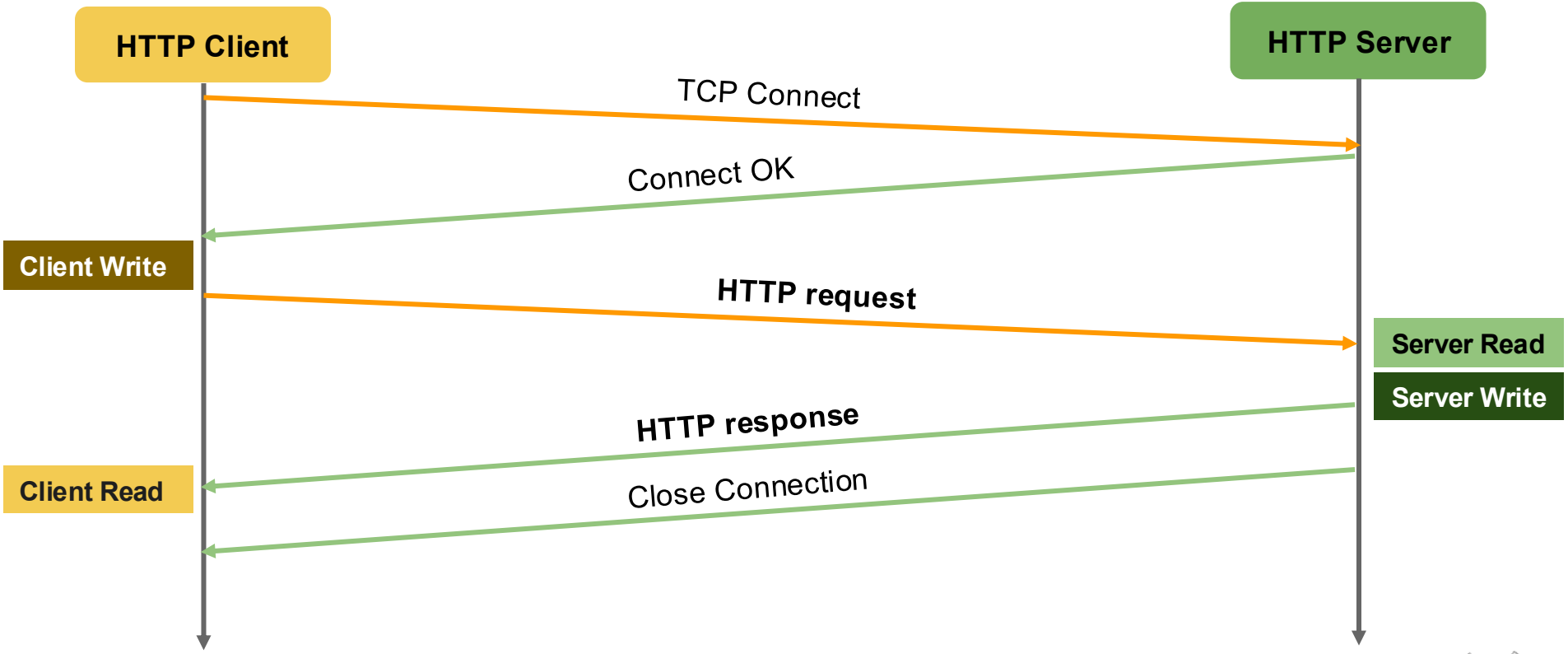
http:// www.gvsu.edu /files/registrar/622GX/7155/ admission.pdf

http:// www.gvsu.edu /files/img/article/frontpag/ 5123FG73A.jpg

http:// www.gvsu.edu /pcec/advising/ index.html



Transaction Timeline (TCP Sockets)



HTTP 1.0 Commands (Request Methods)

- GET
- POST
- HEAD

More-frequently used

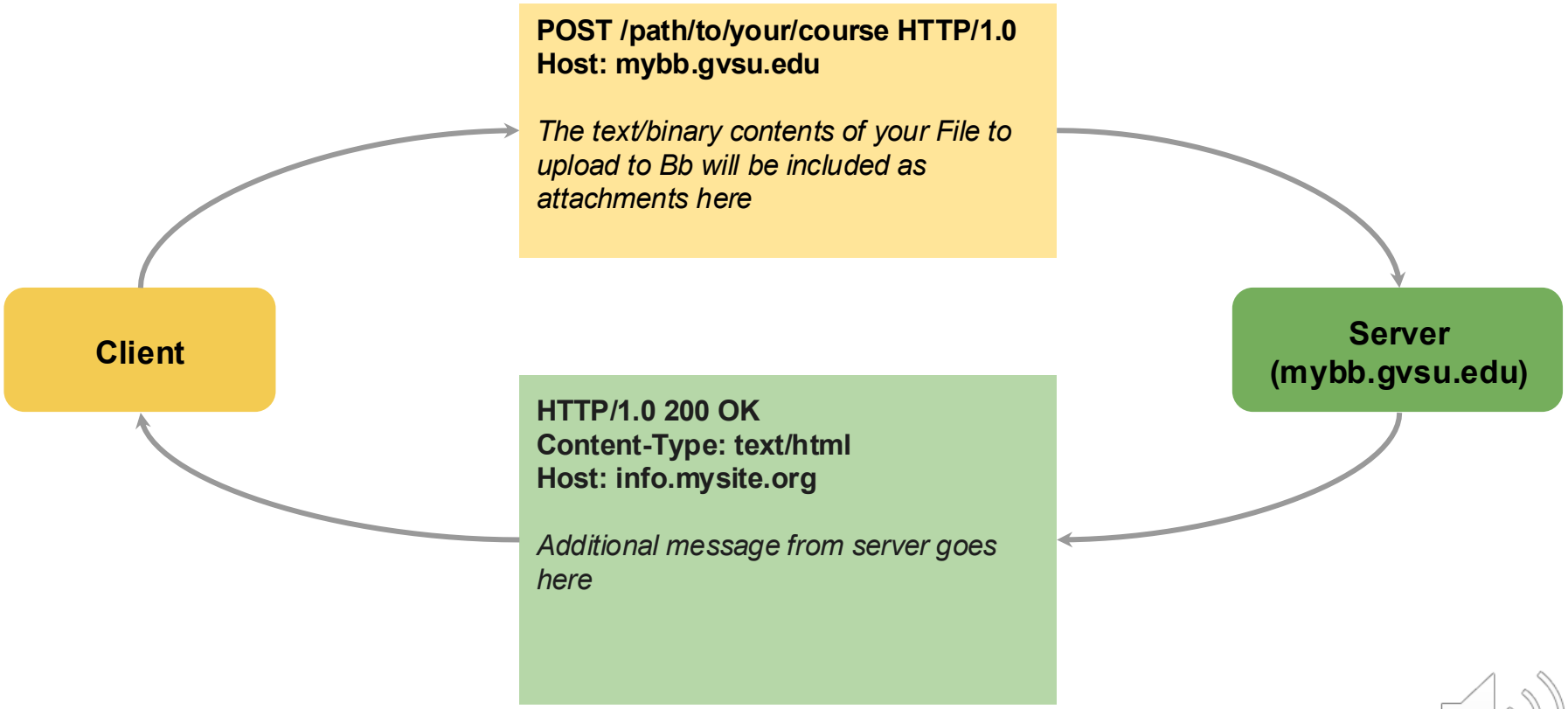
(like GET but the server responds only with header, no data)

- PUT
- DELETE
- OPTIONS

Less-frequently used

Operation	HTTP Request
Create	POST
Read	GET
Update	PUT
Delete	DELETE

POST: upload file to Bb



HTTP Status Code

Status Code	Description
1xx	Informational messages
2xx	Success messages
3xx	Redirect message
4xx	Error on the client's behalf
5xx	Error on the server's behalf

You typed wrong URL → ?

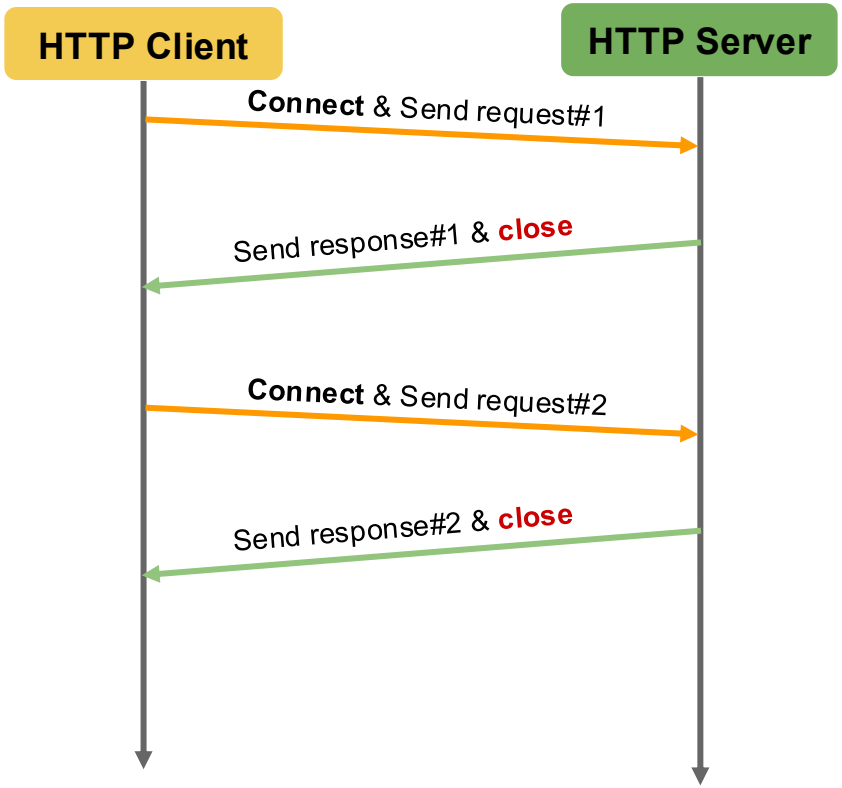
Server crashed → ?

Login required → ?

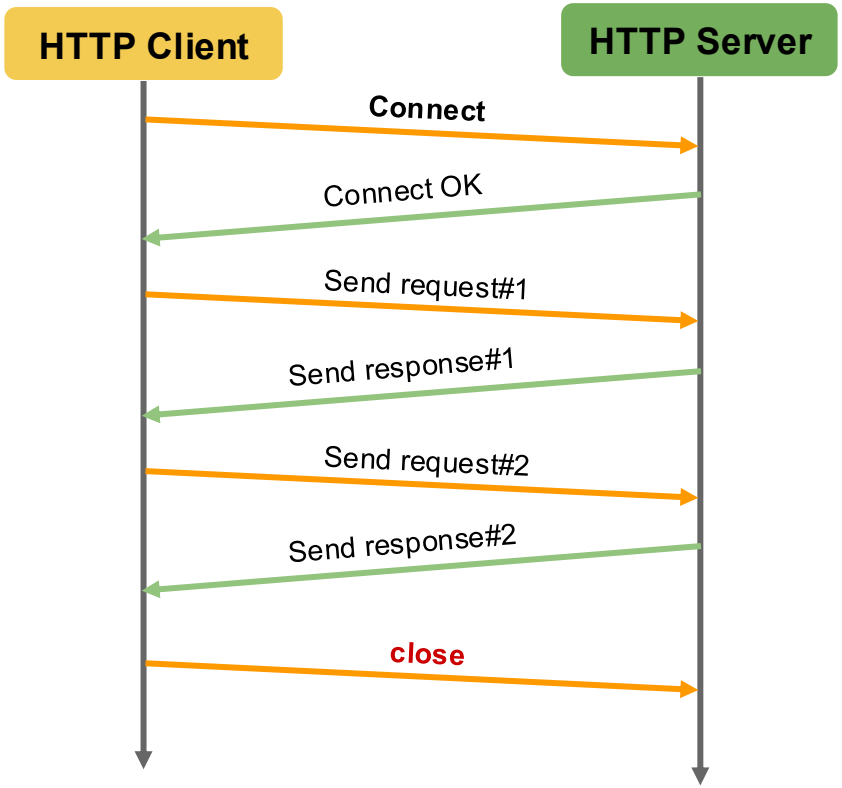
Page moved permanently → ?

Everything OK → ?

HTTP Connections: Persistence



HTTP 1.0: non-persistent



HTTP 1.1: persistent

HTTP 1.0 vs. HTTP 1.1

HTTP 1.0

- One request per connection (non-persistent)
- Cache control is **timestamp based** with one-second resolution (inaccurate)
- Client cannot request a portion of a resource
- Responses are delivered in one big chunk

HTTP 1.1

- N requests per connection (persistent)
- Response can be delivered in chunk
- Cache control is **content based**, responses include entity tag (Etag), similar to hash value
- Clients can request **partial content**
  - "Range:" header line in HTTP request
- Responses may be delivered in many small chunks

HTTP 1.1 vs. HTTP 2

HTTP 1.1

- HTTP messages encoded in text format
- Require multiple connections to achieve concurrency
- Uncompressed response headers
- No resource prioritization

HTTP 2

- HTTP messages encoded in binary format
  - Message = request or response
- Multiple concurrent channels on a single connection
- Compressed response headers
- Resource prioritization (important requests complete more quickly)

HTTP 2 vs. HTTP 3

HTTP 2	HTTP 3
• TCP • Encryption optional	• QUIC (runs on UDP) • Built-in and mandatory